Efficient Attention for Async LLM Training Task

Abstract

Asynchronous decoding with shared context requires attention to preserve the historical visibility of each query. We describe an operator that groups queries by KV-block version, evaluates the groups with a FlashAttention-4 (FA4) variable-length kernel, and combines partial outputs using log-sum-exp (LSE). Query-relative rotary position embeddings (RoPE) allow keys to be rotated once per version. We give the forward and backward merge equations and report isolated forward-plus-backward measurements on NVIDIA H200 using a recorded ViZDoom rollout. The comparison uses a chunked FP32 PyTorch baseline and BF16 FA4 attention; each latency entry is a single measured sample.

FA4 supports custom masks, variable-length batches, and Hopper/Blackwell. Forward returns log-sum-exp (LSE); backward accepts its gradient. The pipeline is:

1. Group queries by KV-block version.
2. Evaluate the groups with an FA4 variable-length kernel.
3. Combine partial outputs with an LSE merge kernel.

1 Example

Generation order:

```
a0 a1 a2 b0 c0 c1 b1 b2 c2 c3
```

A is a shared prefix; B/C decode asynchronously (Hogwild) with mutual context access. Thus `c0/c1` see `b0`; `b1/b2` also see `c0/c1`.

Packed order: `[a0 a1 a2 | b0 b1 b2 | c0 c1 c2 c3]`. Figure 1 preserves execution-order causality, excluding future `b1/b2` from `c0/c1`.

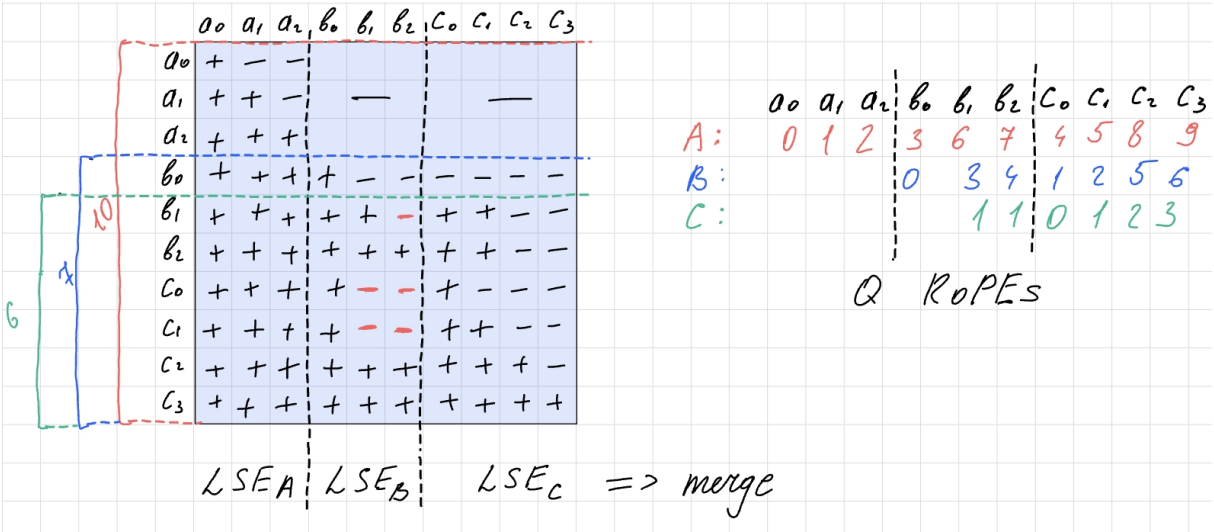


Figure 1. Historical visibility and query RoPE positions for `cache_view=[A,B,C]`.

2 KV blocks and versions

Block-local key positions: A/B use `[0,1,2]`; C uses `[0,1,2,3]`.

Prefill/decode extends the current version; each read records its historical [left, right) range.

Clear, `append_block`, and merge create versions. Prior reads retain their versions; inherited K/V may repeat.

3 Grouping and FA4 varlen

Queries are grouped by KV version; Q_A contains all readers of A.

KV group	Queries	Q / K count
A	`a0 a1 a2 b0 b1 b2 c0 c1 c2 c3`	10 / 3
B	`b0 b1 b2 c0 c1 c2 c3`	7 / 3
C	`b1 b2 c0 c1 c2 c3`	6 / 4

10 tokens yield 23 query copies with individual RoPE positions and masks. Referenced KV spans are packed once per version; repeated reads are preserved.

One varlen call evaluates the three groups with independent masks and softmax. Full score/weight matrices S/P are not materialized; only index plans are cached.

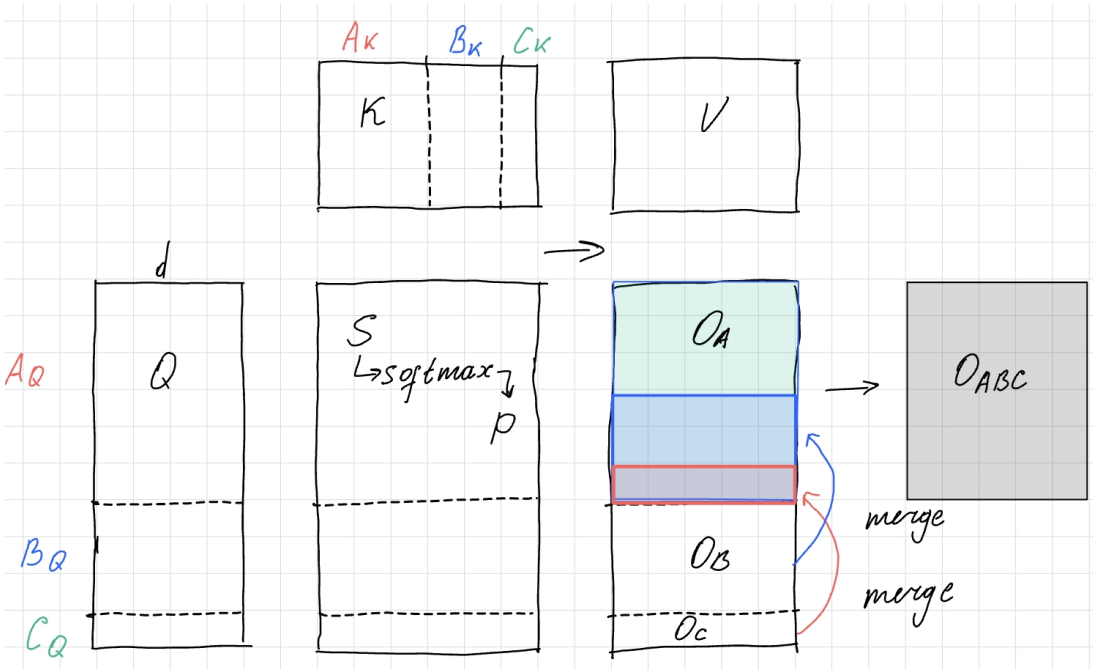


Figure 2. Query packing and partial-output merge (not to scale).

4 Query-relative RoPE

For query q, p denotes its position, s_g block g's offset in its cache view, and j the local key position:

$$\langle R(p - s_g)q, R(j)k\rangle = \langle R(p)q, R(s_g + j)k\rangle. \tag{1}$$

Keys are rotated once per version; query copies absorb view offsets.

For text decode in Figure 1, $p = |A| + |B| + |C| - 1$, using visible lengths after the current write. Block offsets are $s_A = 0$, $s_B = |A|$, $s_C = |A| + |B|$. mRoPE uses recorded 3D coordinates.

5 LSE merge

For one original query q and head: g indexes a KV-block read, j a local key, M_g the visible indices, and d the head dimension.

Variables	Space
s_{gj}, Z_g, ℓ_g, L	$\mathbb{R}$
q_g, k_{gj}, V_j, O_g, O	$\mathbb{R}^d$

5.1 Forward

Partial attention: FA4. s_{gj} is a pre-softmax score of rotated Q/K, scaled by α:

$$s_{gj} = \alpha q_g^\top k_{gj}. \tag{2}$$

Local normalizer Z_g, LSE ℓ_g (natural logarithm), and output O_g, with block values V_j:

$$\begin{aligned} Z_g &= \sum_{j \in M_g} e^{s_{gj}}, \\ \ell_g &= \log Z_g, \\ O_g &= \frac{\sum_{j \in M_g} e^{s_{gj}} V_j}{Z_g}. \end{aligned} \tag{3}$$

Merge (online softmax). L and O are the global LSE and output:

$$\begin{aligned} L &= \operatorname{logsumexp}_g \ell_g, \\ O &= \sum_g e^{\ell_g - L} O_g. \end{aligned} \tag{4}$$

Merge aligns by original query: `a0` uses A, `b0` A+B, `b1` A+B+C.

N counts original queries, N_p copies, and H query heads; Figure 2 has $N_p = 23$, $N = 10$.

Tensor	Direction	Shape
parts: O_g	FA4 $\to$ merge	$N_p \times H \times d$
LSE: ℓ_g	FA4 $\to$ merge	$H \times N_p$
O_{merge}	merge $\to$ next layer	$N \times H \times d$

5.2 Backward

Let $dX = \partial \mathcal{J} / \partial X$ for loss $\mathcal{J}$. Given upstream $dO \in \mathbb{R}^d$, merge computes:

$$\begin{aligned} dO_g &= e^{\ell_g - L} dO \in \mathbb{R}^d, \\ d\ell_g &= e^{\ell_g - L} \langle dO, O_g - O \rangle \in \mathbb{R}. \end{aligned} \tag{5}$$

dO_g is the partial-output gradient; $d\ell_g$ accounts for normalization across blocks.

Gradient: merge $\to$ FA4	Shape	FA4 backward targets
$d$parts: dO_g	$N_p \times H \times d$	Q/K/V
`dLSE`: $d\ell_g$	$H \times N_p$	Q/K; LSE is independent of V

RoPE backward maps Q/K gradients to unrotated coordinates; gather backward accumulates duplicate contributions into the original Q/K/V.

6 Performance

6.1 Workload

The workload is one ViZDoom **Health Gathering** rollout collected with Qwen3.5-27B: 1,392 environment actions and 718,798 recorded tokens. An asynchronous descriptor and thinker provide context to an action-selection probe. The logical KV layout is:

```
Prompt | History | Diff | ImagePair(previous, current)
       | Descriptor | Thinker | Probe
```

ImagePair stores both frames in one KV block; role sub-blocks are abbreviated. Queries select different views of these blocks. Each action adds an observation/thought/action/outcome record to Diff (Delta in the trace). Every 100 actions, `append_block(History, Diff)` extends History, then Diff is cleared. Images and temporary role blocks are refreshed each round.

Benchmarks use initial prefixes ending at complete recorded batches. N counts recorded tokens; $C_{\max}$ is the maximum number of visible keys per query. The largest tested prefix has $N = 65{,}437$ and $C_{\max} = 6{,}533$.

6.2 Measurement protocol

We measure one isolated attention operator, forward plus backward (F+B), including packing, RoPE, attention, and merge. Model projections, other layers, loss, and optimizer updates are excluded. In the tables, **FA4** denotes this complete operator path.

Each backend uses 10 F+B warmups followed by one measured F+B on NVIDIA H200, with checkpointing disabled. Times are synchronized wall time; metadata construction, JIT, input generation, and gradient clearing are excluded. Backend pairs use identical synthetic BF16 Q/K/V and upstream gradients (seed 197), with recorded visibility and mRoPE. Geometries run on separate GPUs; paired backends run sequentially on the same GPU. OMP/MKL and Torch intra-op threads are set to 1.

The `torch_optimized` baseline shares K/V preparation across 128-query chunks and computes attention in FP32 with TF32 disabled. FA4 uses BF16 attention with FP32 statistics and merge accumulation. Reported ratios therefore include arithmetic and implementation differences.

6.3 Isolated latency

Speedup is $T_{\text{Torch}}/T_{\text{FA4}}$. H and H_{KV} denote query and KV head counts; d is the head dimension. Model names specify attention geometry; no model weights are loaded.

Table 1. F+B wall time (ms), Qwen3.5-27B geometry: $H = 24$, $H_{\text{KV}} = 4$, $d = 256$.

N	$C_{\max}$	Torch optimized (ms)	FA4 (ms)	Speedup
1,956	805	54.37	3.33	16.35×
4,008	985	113.66	5.78	19.66×
8,144	1,361	241.28	11.96	20.17×
16,240	2,109	495.41	26.59	18.63×
32,758	3,602	1305.22	65.49	19.93×
65,437	6,533	3995.30	160.07	24.96×

Table 2. F+B wall time (ms), Qwen3.5-0.8B geometry: $H = 8$, $H_{\mathrm{KV}} = 2$, $d = 256$.

N	$C_{\max}$	Torch optimized (ms)	FA4 (ms)	Speedup
1,956	805	49.49	2.98	16.61×
4,008	985	109.32	3.17	34.50×
8,144	1,361	242.57	4.76	50.95×
16,240	2,109	506.62	9.43	53.70×
32,758	3,602	995.85	22.87	43.55×
65,437	6,533	2653.59	55.77	47.58×

All isolated cases completed without OOM. Each entry is one measured sample; no confidence interval is estimated. Ratios use unrounded times. Results apply to this trace's visibility structure, not arbitrary workloads of equal N.

6.4 GPU-time breakdown

A separate Nsight Systems capture measures the FA4 path for the 27B geometry at $N = 65{,}437$, after 10 warmups, with checkpointing disabled. This prefix contains 1,014 KV versions, $N_p = 363{,}714$ query copies, and 70,098 packed KV rows. The capture includes one complete F+B and the `dLSE` path.

Table 3. Sum of GPU kernel durations (ms). Shares are normalized by the total kernel duration, excluding memory operations, CPU time, and timeline gaps.

Component	Forward (ms)	Backward (ms)	Total (ms)	Share (%)
FA4	17.85	89.42	107.27	66.75
LSE merge	2.37	6.04	8.41	5.23
Packing / RoPE / other	17.45	27.56	45.01	28.01
Total	37.67	123.03	160.70	100.00

CUDA launch correlation and NVTX ranges assign all 152 kernels exactly once. The FA4 category includes its preparation and gradient-processing kernels. Memcpy/memset account for a separate 0.013 ms. Totals and shares use unrounded durations.

Profiled wall time is 165.43 ms, including tracing overhead; Table 1 uses the independent, unprofiled 160.07 ms measurement. The merge contribution is 5.23% of summed GPU kernel time for this workload.

Runtime: PyTorch 2.13.0+cu130, CUDA 13.0, FA4 4.0.0b37, CUTLASS DSL 4.6.2. Timing and profiling runs use matching FA4 backend, plan, mask, and merge sources.